BÁO CÁO ĐỒ ÁN – ĐỢT 1

Written by : Nguyen Minh Quan

Contents

Phần 1 : Giới thiệu 2
Phần 2 : 3
Xử lí dữ liệu để tối ưu lưu trữ: 3
Phần 3 : Cách tiếp cận phương pháp chunking 4
Chunk được tạo ra như thế nào? 4
Cơ chế xử lí : 4
1. Chunker — cơ chế chia dữ liệu 4
4. ChunkIndex — cơ chế tra cứu chính xác 5
5. ChunkStore — cơ chế điều phối lưu trữ 6
6. Disk Storage — cơ chế lưu vật lí xuống ổ đĩa 7
8. Garbage Collection (GC) — cơ chế dọn rác 7
9. Read Chunk — cơ chế đọc ngược từ vật lí 8

Phần 1 : Giới thiệu

Layer 1 không làm việc với “tên file logic” hay API người dùng, mà làm việc với **dữ liệu vật lí**. Từ cách các module đang phối hợp, có thể dựng lại mục tiêu cốt lõi của Layer 1 như sau:

- nhận dữ liệu từ file hoặc từ cả thư mục,
- biến dữ liệu thành các đơn vị nhỏ hơn để có thể lưu trữ (nhằm tối ưu hệ thống lưu trữ)
- gán cho mỗi đơn vị một định danh nội dung (cho việc tìm kiếm / quản lí dữ liệu)
- quyết định chunk nào là mới, chunk nào đã tồn tại (ngăn chặn nhân bản)
- ghi chunk mới xuống disk
- giữ một chỉ mục vật lí để biết chunk nằm ở đâu và đang được bao nhiêu file tham chiếu,
- trả ra manifest để Layer 2 biết một file được tạo bởi chuỗi chunk nào theo đúng thứ tự.

  (truyền output cho layer 2)

Phần 2 : Tìm kiếm thuật toán

Xử lí dữ liệu để tối ưu lưu trữ:

Cần 1 thuật toán để biến một file lớn thành nhiều khối nhỏ, để từ đó:

- hệ thống có thể quản lí lưu trữ ở mức nhỏ hơn file,
- có thể deduplicate từng phần dữ liệu,
- lưu trữ dữ liệu vào 1 danh sách để tiết kiệm tài nguyên lưu trữ

Giải pháp : Single blob / Chunk IDs

So sánh :

Đặc điểm	**Single Blob**	**Chunk IDs List**
Dung lượng	Lãng phí nếu có dữ liệu trùng	Tối ưu nhờ Deduplication
Tốc độ sửa đổi	Chậm (phải xử lý cả khối)	Nhanh (chỉ xử lý chunk thay đổi)
Băng thông	Tốn kém khi truyền lại	Tiết kiệm (chỉ truyền chunk mới)
Độ phức tạp	Thấp	Cao (cần quản lý metadata/index)

Dựa vào những đặc điểm trên, nhằm tối ưu hóa hệ thống lưu trữ cho đồ án, nhóm Tài-Quân coop quyết định sử dụng phương pháp Chunking

Phần 3 : Cách tiếp cận phương pháp chunking

Chunk được tạo ra như thế nào?

Tham khảo từ nguồn advanced chunked-base data, Securing the Cloud Storage by Using Different Algorithms of Cryptography

Trước đây, lưu trữ dữ liệu được sử dụng bởi hình thức cổ điển, đó là single blob, chúng xuất hiện các vấn đề như sau :

- **Dư thừa dữ liệu:** Việc lưu trữ khối lượng lớn thông tin gây lãng phí không gian và gây khó khăn trong quản lý.
- **Lỗ hổng bảo mật:** Các phương pháp hiện nay thường gặp lỗi bảo mật trong các kịch bản động hoặc không thể loại bỏ các khối dữ liệu trùng lặp trong quá trình xác thực

Nghiên cứu phát triển một phương pháp khử trùng lặp ở phía người dùng (client-side) để tối ưu hóa không gian và bảo mật:

- **Phân đoạn dữ liệu (CDC):** Sử dụng kỹ thuật *Content-Defined Chunking* để chia nhỏ dữ liệu thành các đoạn (chunks) và gắn thẻ (tags) tương ứng.
- **Kiểm tra trùng lặp:** Thực hiện kiểm tra độ tương đồng dựa trên tên tệp, thuộc tính, kích thước và trọng số đa tương đồng (weighted multi-similarity).
- **Loại bỏ dữ liệu lặp:** Nếu phát hiện trùng lặp, hệ thống sẽ gửi cảnh báo và loại bỏ các tệp lặp lại bằng cách sử dụng kiểm tra tương đồng dựa trên đoạn và mã hóa parity.

Cơ chế xử lí :

1. Chunker — cơ chế chia dữ liệu

Nhận dữ liệu gốc và cắt thành nhiều khối nhỏ.

Cơ chế xử lí cơ bản

- đọc file hoặc nhận buffer bytes,
- duyệt từ đầu đến cuối dữ liệu,
- mỗi lần lấy ra một đoạn có độ dài chunk_size,
- đưa từng đoạn đó thành một chunk.

Kết quả

Một file lớn được biến thành:

$$[chunk_1, chunk_2, chunk_3, \ldots, chunk_n]$$

2. Hashing — cơ chế nhận diện nội dung

Tạo mã định danh cho từng chunk dựa trên chính nội dung của chunk.

<u>Cơ chế xử lí cơ bản</u>

- lấy bytes của một chunk,
- chạy hàm băm,
- sinh ra một chuỗi ID duy nhất, gọi là ChunkID.

<u>Ý tưởng cốt lõi</u>

- cùng nội dung → cùng ChunkID,
- khác nội dung → ChunkID khác.

<u>Kết quả</u>

Mỗi chunk không còn được nhận diện bằng tên file nữa, mà bằng:

$$chunk \rightarrow ChunkID$$

3. Bloom Filter — cơ chế kiểm tra nhanh

Trả lời nhanh câu hỏi:
"chunk này có khả năng đã tồn tại chưa?"

<u>Cơ chế xử lí cơ bản</u>

- khi có một ChunkID, hệ thống dùng nhiều hàm băm,
- mỗi hàm băm cho ra một vị trí trong mảng bit,
- nếu thêm chunk vào Bloom thì các bit đó được bật lên 1,
- khi kiểm tra:
    - nếu có bit nào đang là 0 → chắc chắn chưa tồn tại,
    - nếu tất cả đều là 1 → có thể đã tồn tại.

<u>Ý nghĩa</u>

Bloom Filter *không kết luận chính xác 100%,* mà chỉ làm bước *sàng lọc nhanh.*

4. ChunkIndex — cơ chế tra cứu chính xác

Lưu thông tin vật lí thật của từng chunk.

<u>Cơ chế xử lí cơ bản</u>

Nó hoạt động như một bảng ánh xạ:

$$ChunkID \rightarrow metadata$$

Trong metadata thường có:

- đường dẫn file chunk trên disk,
- kích thước chunk,
- ref_count.

<u>Khi xử lí</u>

- nếu cần biết chunk có tồn tại thật không → tra index,
- nếu cần đọc chunk ở đâu → tra index,
- nếu cần tăng/giảm tham chiếu → sửa metadata trong index.

5. ChunkStore — cơ chế điều phối lưu trữ

Đây là nơi ghép các bước lại thành một pipeline hoàn chỉnh.

<u>Cơ chế xử lí cơ bản khi lưu một chunk</u>

1. nhận chunk,
2. hash để tạo ChunkID,
3. hỏi Bloom Filter,
4. nếu cần thì tra ChunkIndex,
5. nếu chunk đã có:
    - không ghi lại xuống disk,
    - chỉ tăng ref_count,
6. nếu chunk chưa có:
    - ghi chunk xuống file vật lí,
    - thêm metadata vào index,
    - thêm vào Bloom Filter.

<u>Bản chất</u>

ChunkStore là bộ điều phối trung tâm của phương pháp chunking.

6. Disk Storage — cơ chế lưu vật lí xuống ổ đĩa

Lưu chunk mới thành file thật trên filesystem.

Cơ chế xử lí cơ bản

- lấy ChunkID,
- sinh ra đường dẫn vật lí từ ID đó,
- tạo thư mục con theo prefix hash,
- ghi bytes của chunk xuống file .bin.

Ví dụ:

$$ab/cd/< chunkid >.bin$$

7. Reference Count — cơ chế đếm số lần sử dụng

Theo dõi một chunk hiện đang được bao nhiêu file logic tham chiếu tới.

Cơ chế xử lí cơ bản

- chunk mới được lưu → ref_count = 1,
- file khác dùng lại chunk đó → ref_count++,
- file bị xóa → ref_count--.

Ý nghĩa

Cho phép nhiều file dùng chung một chunk vật lí mà không cần lưu lặp lại.

8. Garbage Collection (GC) — cơ chế dọn rác

Xóa những chunk không còn ai dùng nữa.

Cơ chế xử lí cơ bản

- duyệt toàn bộ ChunkIndex,
- tìm chunk nào có ref_count = 0,
- xóa file chunk trên disk,
- xóa metadata khỏi index,
- cập nhật lại trạng thái hệ thống.

Ý nghĩa

Giúp storage không giữ lại rác vật lí sau khi file logic đã bị xóa

9. Read Chunk — cơ chế đọc ngược từ vật lí

Lấy lại dữ liệu chunk từ ChunkID.

Cơ chế xử lí cơ bản

- nhận ChunkID,
- tra ChunkIndex,
- tìm path vật lí,
- đọc bytes từ file chunk,
- kiểm tra lại tính đúng đắn nếu cần,
- trả dữ liệu về cho tầng trên.

Ý nghĩa

Cho phép Layer 2 hoặc tầng phục hồi file lấy lại đúng dữ liệu đã lưu.

10. Manifest — cơ chế nối Layer 1 với Layer 2 (lấy cảm hứng từ lưu trữ phân tán trong Minecraft)

Mô tả một file logic bằng danh sách chunk theo đúng thứ tự.

Cơ chế xử lí cơ bản

Sau khi một file được chia và lưu:

- mỗi chunk sinh ra một ChunkID,
- các ChunkID được ghi lại theo đúng thứ tự xuất hiện trong file,
- cùng với path file và kích thước file.

Kết quả

Layer 2 sẽ nhận được dạng:

$$file \rightarrow [ChunkID_1, ChunkID_2, \ldots, ChunkID_n]$$

Ý nghĩa

Đây là cầu nối giữa:

- dữ liệu vật lí ở Layer 1,
- metadata logic ở Layer 2.

Nguồn Tham Khảo :

Các bài báo liên quan đến chủ đề, trên site Research Gate & Springer Nature Link

Tựa đề :

- **Enhanced authentication for de-duplication of big data on cloud storage system using machine learning approach - Published: 03 November 2023**
- **Securing the Cloud Storage by Using Different Algorithms of Cryptography**
- **Advanced chunk-based data deduplication framework for secure data storage in cloud using hybrid heuristic assisted optimal key-based encryption**